

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 10

Student Name: T. Akshaya

Roll No.: A24126552122

Date: 02-10-2026

Title

Develop a Full-Stack Web Application using Express.js, ReactJS and MongoDB

Aim

To develop a simple **Student Management System** by integrating ReactJS for the frontend, Express.js and Node.js for the backend, and MongoDB for data storage.

Objectives

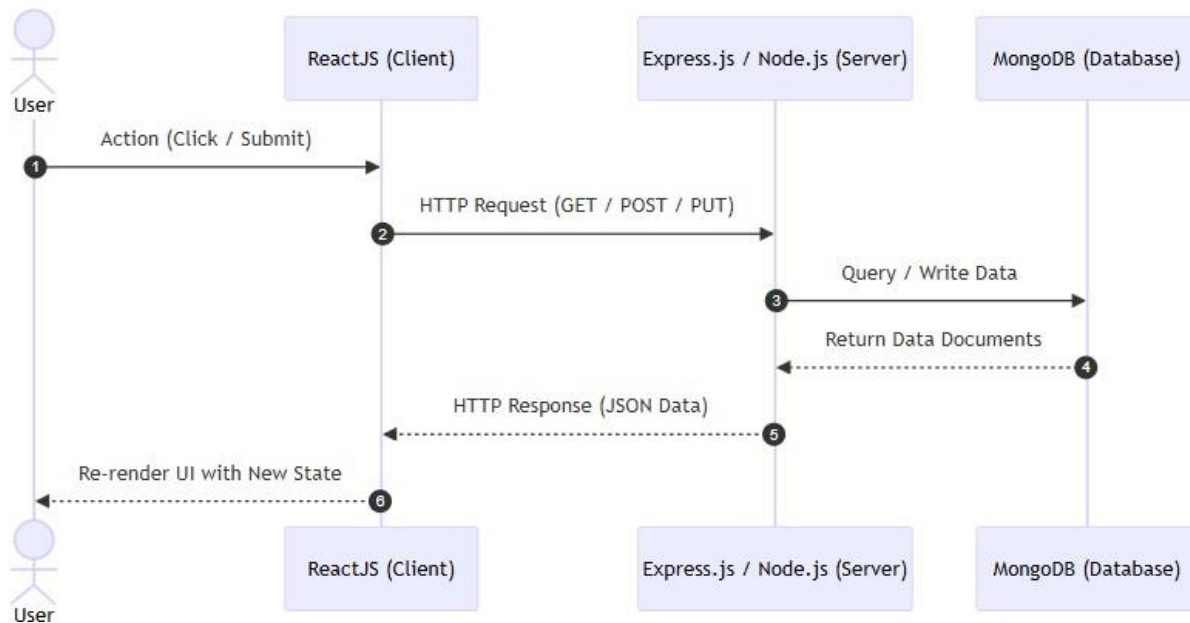
1. To develop a user interface using ReactJS.
2. To create REST API endpoints using Express.js.
3. To connect the backend application with MongoDB.
4. To perform CRUD operations through the web interface.
5. To understand communication between frontend, backend, and database.
6. To test the complete full-stack application.

Theory

A **full-stack web application** consists of three major layers:

- **Frontend:** ReactJS provides the user interface.
- **Backend:** Node.js and Express.js handle HTTP requests and application logic.
- **Database:** MongoDB stores student information.

MERN Stack Request & Response Flow



For this experiment, the application provides the following operations:

Operation	HTTP Method	Purpose
Create	POST	Add a student
Read	GET	Display students
Update	PUT	Modify student details
Delete	DELETE	Remove a student

Algorithm / Procedure

1. Install Node.js, npm, MongoDB and MongoDB Compass.
2. Create separate `backend` and `frontend` folders.
3. Initialize the Node.js backend project.
4. Install Express.js, MongoDB and CORS packages.

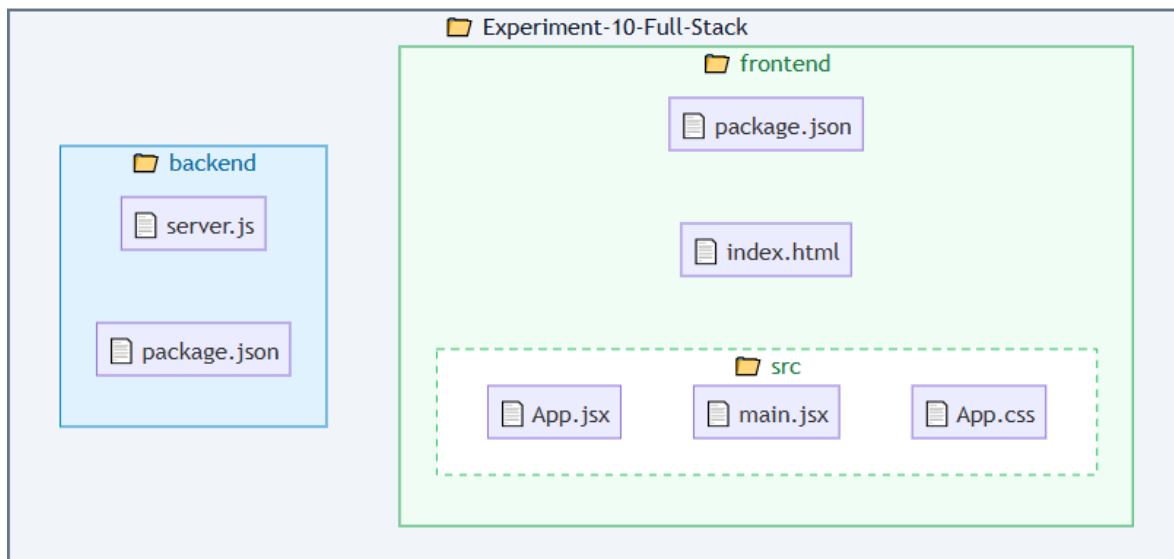
5. Create an Express.js server.
6. Connect the backend to MongoDB.
7. Create REST API endpoints for student CRUD operations.
8. Create a ReactJS frontend using Vite.
9. Create a form to enter student details.
10. Use JavaScript `fetch()` to communicate with the Express API.
11. Display student records retrieved from MongoDB.
12. Implement Add, Update and Delete operations.
13. Start MongoDB, backend server and React frontend.
14. Test the complete application through the browser.

Code Files

Step 1: Folder Structure

Create the following structure:

Project Architecture: Experiment-10-Full-Stack



Step 2: Backend

File 1: `backend/package.json`

```
{
  "name": "student-management-backend",
  "version": "1.0.0",
  "description": "Full Stack Student Management System",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.21.2",
    "mongodb": "^6.12.0"
  }
}
```

File 2: backend/server.js

```
const express = require("express");
const cors = require("cors");
const { MongoClient, ObjectId } = require("mongodb");

const app = express();

const PORT = 5000;
const MONGO_URL = "mongodb://localhost:27017";
const DB_NAME = "collegeDB";

app.use(cors());
app.use(express.json());

let studentsCollection;

// Connect to MongoDB
async function connectDatabase() {

  const client = new MongoClient(MONGO_URL);

  await client.connect();
```

```
    console.log("Connected to MongoDB");

    const database = client.db(DB_NAME);

    studentsCollection = database.collection("students");
}

// GET: Display all students
app.get("/students", async (req, res) => {

    try {

        const students = await studentsCollection
            .find()
            .toArray();

        res.json(students);

    } catch (error) {

        res.status(500).json({
            message: "Error retrieving students"
        });

    }
});

// POST: Add a student
app.post("/students", async (req, res) => {

    try {

        const student = {
            name: req.body.name,
            branch: req.body.branch,
            marks: Number(req.body.marks)
        };

        const result =
```

```

        await studentsCollection.insertOne(student);

res.status(201).json({
  message: "Student added successfully",
  id: result.insertedId
});

} catch (error) {

  res.status(500).json({
    message: "Error adding student"
  });

}
});

// PUT: Update a student
app.put("/students/:id", async (req, res) => {

  try {

    const id = new ObjectId(req.params.id);

    const updatedStudent = {
      name: req.body.name,
      branch: req.body.branch,
      marks: Number(req.body.marks)
    };

    const result =
      await studentsCollection.updateOne(
        { _id: id },
        { $set: updatedStudent }
      );

    res.json({
      message: "Student updated successfully",
      modifiedCount: result.modifiedCount
    });
  }
});

```

```

    } catch (error) {

        res.status(500).json({
            message: "Error updating student"
        });

    }
});

// DELETE: Delete a student
app.delete("/students/:id", async (req, res) => {
    try {

        const id = new ObjectId(req.params.id);

        const result =
            await studentsCollection.deleteOne({
                _id: id
            });

        res.json({
            message: "Student deleted successfully",
            deletedCount: result.deletedCount
        });

    } catch (error) {

        res.status(500).json({
            message: "Error deleting student"
        });

    }
});

// Start server after connecting to MongoDB
connectDatabase()
    .then(() => {

```

```

    app.listen(PORT, () => {

        console.log(
            `Server running at http://localhost:${PORT}`
        );

    });

})
.catch(error => {

    console.error(
        "Unable to connect to MongoDB:",
        error
    );

});

```

File 3: frontend/src/App.jsx

```

import { useEffect, useState } from "react";
import "./App.css";

const API_URL = "http://localhost:5000/students";

function App() {

    const [students, setStudents] = useState([]);

    const [name, setName] = useState("");
    const [branch, setBranch] = useState("");
    const [marks, setMarks] = useState("");

    const [editId, setEditId] = useState(null);

    // GET students
    const getStudents = async () => {

```

```

const response = await fetch(API_URL);

const data = await response.json();

setStudents(data);
};

// Load students when application starts
useEffect(() => {

  getStudents();

}, []);

// Add or Update student
const handleSubmit = async (event) => {

  event.preventDefault();

  const student = {
    name: name,
    branch: branch,
    marks: Number(marks)
  };

  if (editId) {
    await fetch(`${API_URL}/${editId}`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(student)
    });
    alert("Student updated successfully");
    setEditId(null);
  } else {
    await fetch(API_URL, {

```

```

        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(student)
    });
    alert("Student added successfully");
}
clearForm();
getStudents();
};

// Load student data into form
const editStudent = (student) => {
    setEditId(student._id);
    setName(student.name);
    setBranch(student.branch);
    setMarks(student.marks);
};

// Delete student
const deleteStudent = async (id) => {
    const confirmation =
        window.confirm(
            "Are you sure you want to delete this student?"
        );
    if (!confirmation) {
        return;
    }

    await fetch(`${API_URL}/${id}`, {
        method: "DELETE"
    });
    alert("Student deleted successfully");
    getStudents();
};

// Clear form
const clearForm = () => {

```

```

    setName("");
    setBranch("");
    setMarks("");
    setEditId(null);
};

return (
  <div className="container">
    <h1>Student Management System</h1>
    <div className="form-container">
      <h2>
        {editId
          ? "Update Student"
          : "Add Student"}
      </h2>

      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Student Name"
          value={name}
          onChange={ (e) =>
            setName(e.target.value)
          }
          required
        />
        <input
          type="text"
          placeholder="Branch"
          value={branch}
          onChange={ (e) =>
            setBranch(e.target.value)
          }
          required
        />
        <input
          type="number"
          placeholder="Marks"
          value={marks}
          onChange={ (e) =>
            setMarks(e.target.value)
          }
        />
      </form>
    </div>
  </div>
);

```

```

    }
    required
  />
  <button type="submit">

    {editId
      ? "Update Student"
      : "Add Student"}
  </button>
  <button
    type="button"
    onClick={clearForm}
  >
    Clear
  </button>
</form>
</div>
<div className="table-container">
  <h2>Student Records</h2>
  <table>
    <thead>
      <tr>
        <th>Name</th>
        <th>Branch</th>
        <th>Marks</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>
      {students.map(student => (
        <tr key={student._id}>
          <td>{student.name}</td>
          <td>{student.branch}</td>
          <td>{student.marks}</td>
          <td>
            <button
              onClick={() =>
                editStudent(student)
              }
            >

```

```

        Update
      </button>
      <button
        onClick={() =>
          deleteStudent(
            student._id
          )
        }
      >
        Delete
      </button>
    </td>
  </tr>
)}
</tbody>
</table>
</div>
</div>
);
}
export default App;

```

Step 3: Install Backend Packages

Open the terminal inside the `backend` folder:

`npm install`

Start the backend: `npm start`

Expected terminal output: Connected to MongoDB
Server running at http://localhost:5000

Step 4: Frontend

Create the React project inside the main Experiment-10 folder:

npm create vite@latest frontend----- template react

Then:

cd frontend

npm install

File 3: frontend/src/App.jsx

File 4: frontend/src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";

import App from "../App.jsx";

createRoot(
  document.getElementById("root")
).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

File 5: frontend/src/App.css

NOTE: CSS is in the soft copy and in GitHub repo

Step 5: Run the Frontend

Open another terminal inside the frontend folder:

npm run dev

Step 6: Execution

The three components should now be running:

Backend Terminal Connected to MongoDB
Server running at http://localhost:5000

Frontend Terminal VITE ready

Local: http://localhost:5173/

http://localhost:5173/

Student Management System

Add Student

Add StudentClear

Student Records

Name	Branch	Marks	Actions
Ravi Kumar	CSE	85	<button>Update</button> <button>Delete</button>
Priya Sharma	AIML	91	<button>Update</button> <button>Delete</button>
Kiran Kumar	CSE	78	<button>Update</button> <button>Delete</button>

Output 1: React frontend displays student records retrieved from MongoDB through Express API.

http://localhost:5173/

Student Management System

Add Student

Add StudentClear

Student Records

Name	Branch	Marks	Actions
Ravi Kumar	CSE	85	<button>Update</button> <button>Delete</button>
Priya Sharma	AIML	91	<button>Update</button> <button>Delete</button>
Kiran Kumar	CSE	78	<button>Update</button> <button>Delete</button>
Anil Kumar	CSE	88	<button>Update</button> <button>Delete</button>

Output 2: POST operation adds Anil Kumar and the new record appears in the React table.

http://localhost:5173/

Student Management System

Update Student

Update StudentClear

Student Records

Name	Branch	Marks	Actions
Ravi Kumar	CSE	90	<button>Update</button> <button>Delete</button>
Priya Sharma	AIML	91	<button>Update</button> <button>Delete</button>
Kiran Kumar	CSE	78	<button>Update</button> <button>Delete</button>
Anil Kumar	CSE	88	<button>Update</button> <button>Delete</button>

Output 3: PUT operation updates Ravi Kumar's marks from 85 to 90.

http://localhost:5173/

Student Management System

Add Student

Add StudentClear

Student Records

Name	Branch	Marks	Actions
Ravi Kumar	CSE	90	<button>Update</button> <button>Delete</button>
Priya Sharma	AIML	91	<button>Update</button> <button>Delete</button>
Kiran Kumar	CSE	78	<button>Update</button> <button>Delete</button>

Output 4: DELETE operation removes Anil Kumar from the MongoDB collection.

http://localhost:5173/

Student Management System

Add Student

Add StudentClear

Student Records

Name	Branch	Marks	Actions
------	--------	-------	---------

Output 5: Final refresh can be used to verify the current records after CRUD operations.

Test Cases & Execution Data

TC No.	Test Case	Input	Expected Output	Result
TC01	Add Student	Ravi, CSE, 85	Student added successfully	Pass
TC02	Add Student	Priya, AIML, 91	Student added successfully	Pass
TC03	Display Students	Click/Load Student Records	All student records displayed	Pass
TC04	Update Student	Change Ravi's marks from 85 to 90	Student details updated	Pass
TC05	Delete Student	Delete Priya	Student removed from records	Pass
TC06	Clear Form	Click Clear	Input fields become empty	Pass